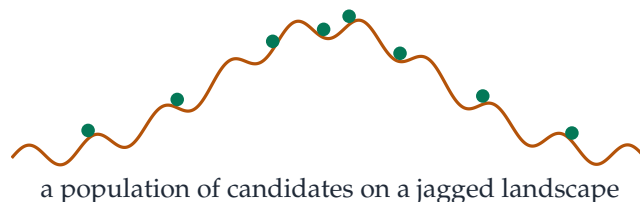

Artificial Intelligence: Search Methods for Problem Solving

Week 4 — Stochastic Local Search: Randomized Algorithms

Detailed study notes built on the lectures of Prof. Deepak Khemani (IIT Madras)

(A First Course in Artificial Intelligence, Chapter 4)

IITM BS in Programming & Data Science



Modules follow the order of the slides. Yellow boxes give intuition, blue boxes are definitions, purple boxes are formulas and derivations, black boxes are algorithms (as on the slides), green boxes are side notes, red boxes flag exam traps and slide errata. Every table and crossover example from the slides was recomputed by code; the few that disagree are flagged.

Contents

1	Recap: Why Randomness?	3
1.1	Exploding search spaces	3
1.2	The SAT problem, precisely	3
1.3	Complexity of SAT: 2SAT, 3SAT, k SAT	4
1.4	Probability of 3SAT being satisfiable: the phase transition	4
1.5	Recap of deterministic escapes	4
2	Stochastic Actions: Random Walk to Simulated Annealing	6
2.1	Random Walk: pure exploration	6
2.2	Stochastic Hill Climbing	6
2.3	The sigmoid acceptance function	6
2.4	Annealing: the physical inspiration	7
2.5	Simulated Annealing	7
2.6	The effect of T	8
2.7	Intuition: why Simulated Annealing works	9
2.8	When Simulated Annealing works well	9
3	Population-Based Methods: Genetic Algorithms	11
3.1	Life on Earth: the inspiration	11
3.2	Towards fitter members: mixing genes	11
3.3	Genetic Algorithms	12
3.4	Crossover on SAT: a 6-variable example	13
3.5	How a population moves on the landscape	14
3.6	A tiny example (Goldberg, 1989)	14
3.7	Practical matters	15
4	Genetic Algorithms for the TSP	16
4.1	Setting up the GA	16
4.2	Single-point crossover does not work	16
4.3	Cycle Crossover (CX)	16
4.4	Partially Mapped Crossover (PMX)	17
4.5	Order Crossover (OX)	17
4.6	Adjacency representation	18
4.7	Ordinal representation	19
5	Emergent Systems: Complex Behaviour from Simple Elements	21
5.1	Conway's Game of Life (1970)	21

5.2	Chaos and fractals	22
5.3	The brain	22
5.4	Karl Sims: evolved virtual creatures (1994)	23
6	Ant Colony Optimization	24
6.1	Biosemitotics: communicating through signs	24
6.2	Ant colonies and foraging	24
6.3	Ant Colony Optimization (ACO)	25
6.4	The transition rule	25
6.5	Updating pheromone	26
Practice: Test Yourself		28
Part A — Numerical problems and traces		28
Part B — Subjective questions		31
Part C — MCQs		34

Module 1 Recap: Why Randomness?

1.1 Exploding search spaces

Week 3 ended with the sizes of two classic candidate spaces:

N	SAT: 2^N	TSP: $N!$	ratio
5	32	120	3.75
8	256	40,320	157.5
50	1.13×10^{15}	3.04×10^{64}	2.7×10^{49}
100	1.27×10^{30}	9.33×10^{157}	7.4×10^{127}

No systematic method can look at more than a vanishing fraction of such spaces. Local search looks at very few candidates; this week makes it **randomised** so it does not get trapped.

1.2 The SAT problem, precisely

Definition: SAT in CNF

Given Boolean (propositional) variables, each taking a value in $\{1,0\}$, and a formula F in **conjunctive normal form**, find a valuation that makes F true.

- Every **clause** must be satisfied.
- A clause is satisfied if **any one** of its literals is satisfied.
- A literal x is satisfied if $x = 1$; a literal $\neg x$ is satisfied if $x = 0$.

Candidates are bit strings: 101110 means $a=1, b=0, c=1, d=1, e=1, f=0$. With N variables there are 2^N candidates (the state space).

Formula / Derivation: Checking the slide's example

$F = (a \vee \neg b \vee e) \wedge b \wedge (\neg c \vee d \vee f) \wedge (d \vee \neg e) \wedge (\neg a \vee b \vee d) \wedge (a \vee \neg d)$ at 101110:

Clause	value	reason
$a \vee \neg b \vee e$	1	$a = 1$
b	0	$b = 0$
$\neg c \vee d \vee f$	1	$d = 1$
$d \vee \neg e$	1	$d = 1$
$\neg a \vee b \vee d$	1	$d = 1$
$a \vee \neg d$	1	$a = 1$

Five of six clauses: $h = 5$. Flipping b gives 111110, which satisfies all six.

1.3 Complexity of SAT: 2SAT, 3SAT, k SAT

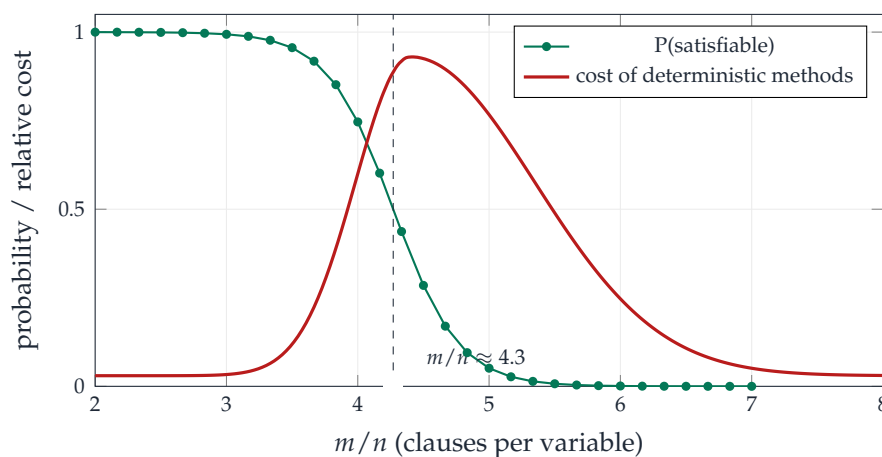
Class	Definition	Complexity
2SAT	every clause has at most two literals, e.g. $(a \vee \neg b) \wedge (\neg a \vee c) \wedge (\neg c \vee f) \wedge \dots$	in P (polynomial time)
3SAT	every clause has at most three literals	NP-complete
kSAT, $k > 3$	clauses of up to k literals	NP-complete; “could be much harder” in practice

Note: Why 2SAT is easy

A clause $(x \vee y)$ is equivalent to the two implications $\neg x \Rightarrow y$ and $\neg y \Rightarrow x$. Build a directed *implication graph* on the $2N$ literals. The formula is unsatisfiable iff some variable x and $\neg x$ lie in the same strongly connected component — checkable in linear time. With three literals per clause, a false literal no longer forces a single consequence, and this trick breaks down.

1.4 Probability of 3SAT being satisfiable: the phase transition

For random 3SAT with n variables and m clauses, the key parameter is the ratio m/n .



Intuition: Under-, critically and over-constrained

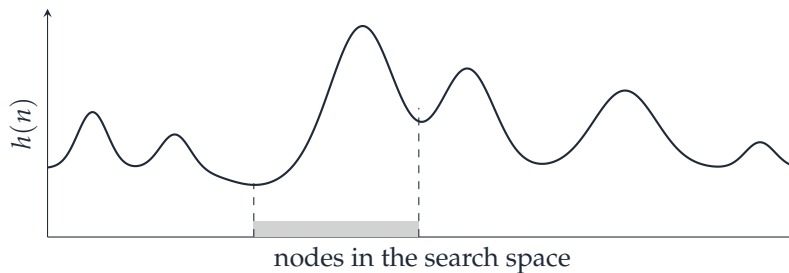
- **Few clauses** (m/n small): almost every random formula is satisfiable, and many assignments work — easy to find one.
- **Many clauses** (m/n large): almost every formula is unsatisfiable, and contradictions show up quickly — easy to prove.
- **Near $m/n \approx 4.26$** : the probability of satisfiability drops from 1 to 0, and deterministic (complete) methods take longest. The hardest instances live at this **phase transition**. The slide’s “4.3” is this value rounded.

Stochastic local search cannot prove unsatisfiability, but on satisfiable instances near the transition it often finds solutions far faster than complete methods.

1.5 Recap of deterministic escapes

- **Tabu search** walks along ridges out of local maxima by forbidding recent moves.

- **Beam search** keeps w candidates per level, but can still collapse onto one local optimum (Week 3's SAT example).
- **Iterated Hill Climbing**: what is the sanctity of the start node? Choose different starting points.



A random start in the shaded basin leads Hill Climbing to the global maximum. Every other start ends on a local maximum.

Formula / Derivation: How many restarts?

If the global maximum's basin covers a fraction p of the space, a random start succeeds with probability p , and k independent restarts all fail with probability $(1 - p)^k$. For success probability $1 - \delta$,

$$k \geq \frac{\ln \delta}{\ln(1 - p)} \approx \frac{1}{p} \ln \frac{1}{\delta} \quad (\text{small } p).$$

Example: $p = 0.1$, $\delta = 0.01$: $k \geq \ln 0.01 / \ln 0.9 = 43.7$, so 44 restarts. Iterated Hill Climbing works when the good basin is not too small; when the landscape is very jagged (p tiny), we need methods that can climb *out* of bad basins — this week's topic.

Module recap

- SAT in CNF: a clause is satisfied by any one true literal; candidates are N -bit strings.
- 2SAT is in P (implication graph); 3SAT is NP-complete; hardest random instances at $m/n \approx 4.26$.
- Iterated Hill Climbing needs $\approx \frac{1}{p} \ln \frac{1}{\delta}$ restarts for a basin fraction p .

Module 2 Stochastic Actions: Random Walk to Simulated Annealing

To move or not to move ... is the question.

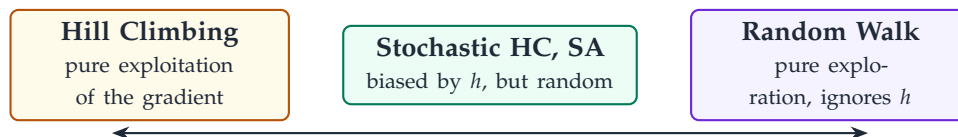
2.1 Random Walk: pure exploration

Algorithm: RandomWalk() — from the slide

```

1: node ← random candidate solution or start
2: bestNode ← node
3: for i ← 1 to n do
4:   node ← RANDOMCHOOSE(MOVEGEN(node))
5:   if node is better than bestNode then
6:     bestNode ← node
7:   end if
8: end for
9: return bestNode

```



Note

Random Walk explores the space in a random fashion and remembers the best node seen; its only use of h is in recording $bestNode$. It never gets stuck, but it also never heads anywhere in particular: on a space of 2^{50} candidates, blind wandering is hopeless. Hill Climbing does the opposite. The rest of the week mixes the two.

2.2 Stochastic Hill Climbing

Definition: Stochastic hill climbing

At the current node v_c , select a **random** neighbour v_N and compute

$$\Delta E = eval(v_N) - eval(v_c) \quad (\text{maximisation}).$$

If $\Delta E > 0$ (v_N is better), move to it with **high** probability. If $\Delta E < 0$ (v_N is worse), move to it with **low** probability.

2.3 The sigmoid acceptance function

Formula / Derivation: Acceptance probability

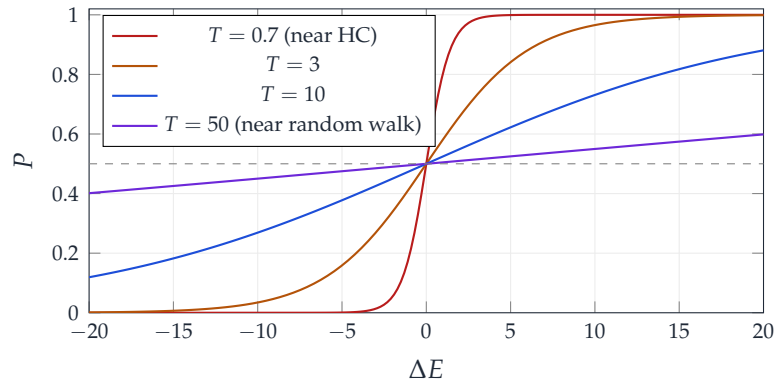
$$P(\text{move}) = \frac{1}{1 + e^{-\Delta E/T}}, \quad \Delta E = eval(v_N) - eval(v_c), \quad T > 0.$$

Properties (each a one-line check):

- $\Delta E = 0$: $P = \frac{1}{1+1} = \frac{1}{2}$.
- $\Delta E \rightarrow +\infty$: $e^{-\Delta E/T} \rightarrow 0$, $P \rightarrow 1$. $\Delta E \rightarrow -\infty$: $e^{-\Delta E/T} \rightarrow \infty$, $P \rightarrow 0$.

- Symmetry: $P(\Delta E) + P(-\Delta E) = \frac{1}{1+e^{-x}} + \frac{1}{1+e^x} = \frac{1}{1+e^{-x}} + \frac{e^{-x}}{e^{-x}+1} = 1$ with $x = \Delta E/T$.
- Only the ratio $\Delta E/T$ matters: doubling T has the same effect as halving every ΔE .

It accepts a *good* random move with high probability and a *bad* one with low probability — combining exploration with exploitation.



2.4 Annealing: the physical inspiration

Definition: Annealing (metallurgy)

A heat treatment that increases the ductility and reduces the hardness of a material, making it more workable. The material is heated above its recrystallisation temperature, held there for a suitable time, then **cooled slowly**. At high temperature atoms migrate in the crystal lattice and the number of dislocations decreases; as the material cools it recrystallises into a low-energy, well-ordered state.

Intuition: The analogy

Physics	Search
configuration of atoms	candidate solution
energy (to be minimised)	— <i>eval</i> (the slides maximise <i>eval</i>)
temperature T	a parameter controlling randomness
hot: atoms move freely	large T : many bad moves accepted (exploration)
cooling slowly	lowering T gradually
frozen crystal	final solution, near the global optimum
quenching (cooling fast)	dropping T fast $\approx$ Hill Climbing: stuck in a local optimum

2.5 Simulated Annealing

Algorithm: SimulatedAnnealing() — begin with exploration, end with exploitation

```

1: node  $\leftarrow$  random candidate solution or start
2: bestNode  $\leftarrow$  node
3:  $T \leftarrow$  some large value
4: for time  $\leftarrow$  1 to numberOfEpochs do
5:   while some termination criteria do
6:     neighbour  $\leftarrow$  RANDOMNEIGHBOUR(node)
7:      $\Delta E \leftarrow$  EVAL(neighbour) — EVAL(node)
8:     if RANDOM(0,1) <  $1/(1 + e^{-\Delta E/T})$  then

```

▷ M cycles in a simple case

```

9:         node ← neighbour
10:        if EVAL(node) > EVAL(bestNode) then
11:            bestNode ← node
12:        end if
13:    end if
14: end while
15: T ← COOLINGFUNCTION(T, time)
16: end for
17: return bestNode

```

Note: Details on the slide

Eval is used instead of *h*, in the style of optimisation. *CoolingFunction* lowers the temperature after each epoch. $\text{RANDOM}(0, 1)$ returns a number uniformly distributed in $[0, 1)$, so the test on line 8 succeeds with exactly the sigmoid probability. *bestNode* is kept because the current node may wander away from the best found.

Formula / Derivation: Common cooling schedules

- **Geometric:** $T_{k+1} = \alpha T_k$ with $0.8 \leq \alpha < 1$. From $T_0 = 100$ with $\alpha = 0.9$: $T_{10} = 34.87$, $T_{20} = 12.16$, $T_{30} = 4.24$, $T_{40} = 1.48$.
- **Linear:** $T_{k+1} = T_k - \Delta$.
- **Logarithmic:** $T_k = c / \ln(1 + k)$ — very slow; theoretical results show SA reaches the global optimum with probability 1 under such schedules, but they are too slow in practice.

2.6 The effect of T

- **Very large T :** $\Delta E / T \approx 0$, so $P \approx 0.5$ for every move — the algorithm behaves like **Random Walk**.
- $T \rightarrow 0$: $P \rightarrow 1$ for $\Delta E > 0$ and $P \rightarrow 0$ for $\Delta E < 0$ — deterministic, like **Hill Climbing**.
- As T decreases the moves become more exploitative.

2.6.1 The slide's tables (from Michalewicz and Fogel, *How to Solve It: Modern Heuristics*, p. 119)

Formula / Derivation: Effect of temperature, for a good move with $\Delta E = +13$

The slide writes $-\Delta E = \text{eval}(v_c) - \text{eval}(v_n) = -13$, i.e. the neighbour is better by 13. Then $P = 1 / (1 + e^{-13/T})$:

T	$e^{-13/T}$	P
1	0.000002	1.00
5	0.0743	0.93
10	0.2725	0.79 (slide: 0.78)
20	0.522	0.66
50	0.771	0.56
10^{10}	0.99999...	0.50

Even a clearly better move is taken only with probability $\approx \frac{1}{2}$ when T is huge.

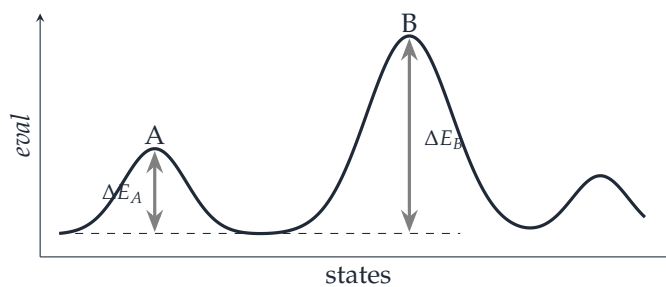
Formula / Derivation: Effect of ΔE , at $eval(v_c) = 107, T = 10$

$$P = 1 / (1 + e^{(eval(v_c) - eval(v_n)) / 10}):$$

$eval(v_n)$	$eval(v_c) - eval(v_n)$	$e^{(\cdot)/10}$	P
80	27	14.88	0.06
100	7	2.01	0.33
107	0	1.00	0.50
120	-13	0.27	0.78
150	-43	0.01	0.99

Worked row: $eval(v_n) = 100$: $e^{0.7} = 2.014$, $P = 1 / 3.014 = 0.332$. The worse the neighbour, the less likely the move.

2.7 Intuition: why Simulated Annealing works



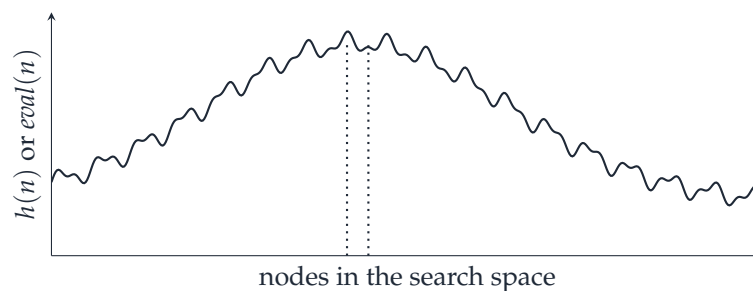
Formula / Derivation: A goes to B more easily than B goes to A

To leave local maximum A for B, the search must first go *down* through the valley: a loss of ΔE_A . To go from B to A it must lose $\Delta E_B > \Delta E_A$. The probability of accepting a loss L is about $e^{-L/T}$ for $L \gg T$ (since $1 / (1 + e^{L/T}) \approx e^{-L/T}$). So

$$\frac{P(A \rightarrow B)}{P(B \rightarrow A)} \approx e^{(\Delta E_B - \Delta E_A) / T}.$$

With $\Delta E_A = 3$, $\Delta E_B = 8$ (single-step losses, computed with the exact sigmoid): at $T = 1$ the ratio is $0.0474 / 0.000335 \approx 141$; at $T = 2$ about 10; at $T = 4$ about 2.7. **At every temperature the search drifts towards higher peaks**, and as T falls it becomes increasingly unlikely to leave the highest one.

2.8 When Simulated Annealing works well



A jagged surface with many local optima but a broader trend towards the optimum. The dotted strip is the tiny footprint of Hill Climbing: its basin of the global maximum.

Intuition

At high T , SA does not “see” the small wiggles (it crosses them easily) and follows the broad trend. As T drops it starts to respect smaller and smaller features, finally climbing the highest local bump. If the landscape has no broad trend (a needle in a haystack), no local method helps.

Note: Metropolis acceptance

Many textbooks use a different rule: always accept an improving move, and accept a worsening move with probability $e^{\Delta E/T}$ (Metropolis). The slides use the sigmoid, which accepts improving moves only with probability $> \frac{1}{2}$. Both share the key features: worse moves are accepted less often, and less often at lower temperature.

Module recap

- Random Walk = pure exploration; Hill Climbing = pure exploitation.
- Stochastic HC picks a random neighbour and accepts it with $P = 1/(1 + e^{-\Delta E/T})$.
- Large $T \Rightarrow P \approx \frac{1}{2}$ (random walk); $T \rightarrow 0 \Rightarrow$ Hill Climbing.
- Simulated Annealing starts hot (explore) and cools (exploit), remembering *bestNode*.
- It prefers climbing out of shallow optima to deep ones; it suits jagged landscapes with a broad trend.

Module 3 Population-Based Methods: Genetic Algorithms

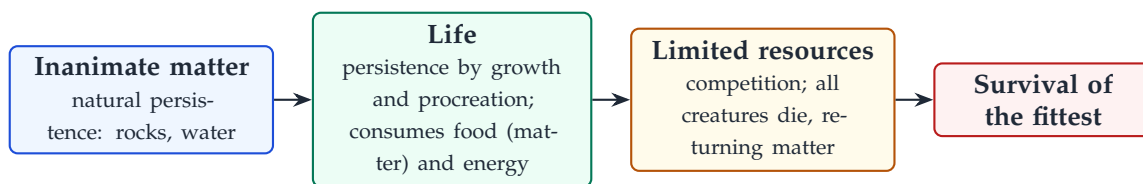
3.1 Life on Earth: the inspiration

The lecture opens with questions: What is life? Why are there different life forms? Is evolution a design process? Is Nature looking for a perfect design, and if so, to what purpose?

Definition: Steve Grand's "most important law of nature"

In *Creation: Life and How to Make It*: **Things that persist, persist. Things that don't, don't.**

Much of what persists — clouds on a hilltop, ripples in a pond, even human bodies — is a persistent *pattern* in a constantly changing substrate: our bodies are made of entirely different atoms from when we were children.



- **Competition** happens between species (lions and hyenas for the same prey) and *within* a species: the fastest or wildest foxes get the rabbits; the fastest or cleverest rabbits escape.
- **Ecosystems**: millions of species interact; an arrow in a food web means the population of one species positively influences another.
- **Evolution**: a species is like the *design* of a life form. Richard Dawkins (*The Blind Watchmaker*) argues that nature's "ratchet mechanism" — things that persist, persist — produced complex life without a designer.

3.2 Towards fitter members: mixing genes

- **Asexual reproduction** (cloning) replicates the parent: no new combinations.
- **Sexual reproduction** inherits features from **two** parents. Every offspring is unique; siblings are not identical.
- Paul Valéry: "It takes two to invent anything. The one makes up combinations, the other chooses..." — mixing genes *generates*, natural selection *tests*: evolution is an instance of **generate and test**.
- Mates are not chosen at random: selection by attractiveness (song-and-dance shows of male birds; for humans, health, wealth, cleverness...).

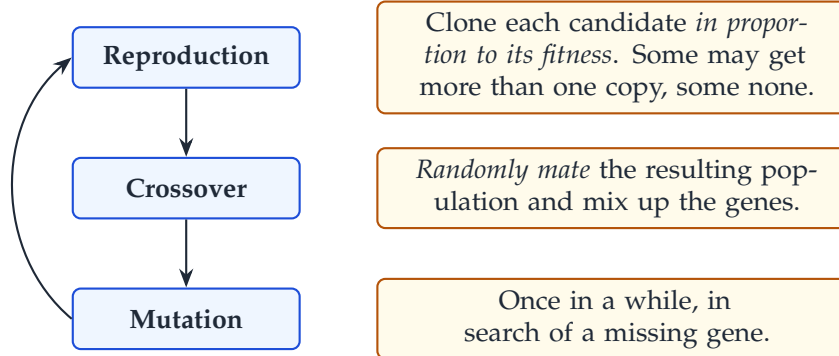
Definition: Genotype and phenotype

An organism's **genotype** is the set of genes in its DNA responsible for a trait. Its **phenotype** is the physical expression of those genes — the entity that actually competes in the world. Selection acts on phenotypes; inheritance passes on genotypes. In a GA: the bit string is the genotype, the decoded candidate and its fitness are the phenotype.

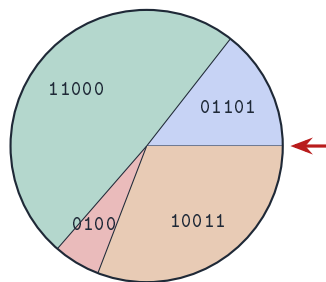
3.3 Genetic Algorithms

Definition: Genetic Algorithm (GA)

Devised by **John Holland** (1975), popularised by his student **David Goldberg** (*Genetic Algorithms in Search, Optimization and Machine Learning*, 1989). A class of **evolutionary algorithms** for optimisation: heuristic, stochastic, adaptive search on a **population** of candidates in the solution space. A **fitness function** evaluates each candidate; the fittest get to mate and reproduce.



3.3.1 Selection: the roulette wheel



Sector angle $\propto$ fitness: $360^\circ \times f_i / \sum f$.
 For Goldberg's example (below): 169, 576, 64, 361 of 1170 give 52° , 177° , 20° , 111° .
 Spin N times; each spin adds one parent to the new population. Members are then paired randomly for crossover.

Formula / Derivation: "How will you implement this in a program?" (the slide's question)

Compute cumulative probabilities $c_i = \sum_{j \leq i} f_j / \sum_k f_k$. For each spin draw $r \sim \text{Uniform}[0, 1)$ and select the first i with $r < c_i$ (binary search makes this $O(\log N)$).

Goldberg's example: $c = (0.144, 0.637, 0.691, 1.000)$. A draw $r = 0.50$ selects 11000; $r = 0.66$ selects 01000; $r = 0.90$ selects 10011.

Expected copies of candidate i in N spins $= N f_i / \sum f = f_i / \bar{f}$ (fitness relative to the average). Fitness must be non-negative; for minimisation (TSP) use e.g. $f = 1/\text{cost}$ or $f = C_{\max} - \text{cost}$.

3.3.2 Crossover operators

Formula / Derivation: Single-point and multi-point crossover

Cut both parents at a random point and swap tails:

$$P_1 = X_1 X_2 X_3 X_4 \mid X_5 X_6 X_7 X_8$$

$$C_1 = X_1 X_2 X_3 X_4 Y_5 Y_6 Y_7 Y_8$$

$$P_2 = Y_1 Y_2 Y_3 Y_4 \mid Y_5 Y_6 Y_7 Y_8$$

$$C_2 = Y_1 Y_2 Y_3 Y_4 X_5 X_6 X_7 X_8$$

Any operator that mixes up the genes will do. With multi-point (here *uniform*, alternating) crossover: $C_3 = X_1 Y_2 X_3 Y_4 X_5 Y_6 X_7 Y_8$, $C_4 = Y_1 X_2 Y_3 X_4 Y_5 X_6 Y_7 X_8$.

For strings of length L there are $L - 1$ cut points for single-point crossover.

3.3.3 The algorithm

Algorithm: Genetic-Algorithm() — from the slide

- 1: $P \leftarrow$ create N candidate solutions ▷ initial population
- 2: **repeat**
- 3: compute fitness value for each member of P
- 4: $S \leftarrow$ with probability proportional to fitness, randomly select N members from P
- 5: $offspring \leftarrow$ partition S into two halves, randomly mate and crossover members to generate N offspring
- 6: with a low probability mutate some offspring
- 7: replace k weakest members of P with k strongest offspring
- 8: **until** some termination criteria
- 9: **return** the best member of P

Note: Line 7: replacement

With $k = N$ the whole population is replaced each generation (generational GA, as in Goldberg's example). Smaller k keeps the best current members (*elitism*), so the best fitness never decreases.

3.4 Crossover on SAT: a 6-variable example

$$F = (a \vee \neg b \vee e) \wedge (\neg a \vee b \vee c) \wedge (\neg c \vee d \vee f) \wedge (d \vee \neg e \vee f) \wedge (\neg a \vee b \vee d) \wedge (a \vee \neg d \vee f),$$

h = number of satisfied clauses; bits in order $abcdef$; single-point crossover after bit 3.

	string	clause by clause (computed)	h	slide
P_1	010110	1 + 1 + 1 + 1 + 1 + 0	5	4
P_2	111010	1 + 1 + 0 + 0 + 1 + 1	4	4
C_1	111110	1 + 1 + 1 + 1 + 1 + 1	6	6
C_2	010010	1 + 1 + 1 + 0 + 1 + 1	5	4

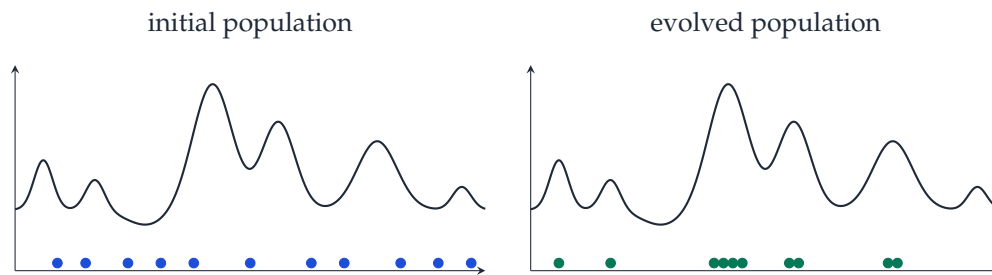
Slide erratum: Clause 1 for P_1 and C_2

The slide scores clause 1, $(a \vee \neg b \vee e)$, as 0 for $P_1 = 010110$ and $C_2 = 010010$. Both have $e = 1$ (fifth bit), so clause 1 is satisfied: P_1 and C_2 have $h = 5$, not 4. The lesson is unaffected: crossover of two imperfect parents produced $C_1 = 111110$, which satisfies all 6 clauses. (F has 30 satisfying assignments out of 64.)

Intuition: Why crossover can work

P_2 has a good “head”: $abc = 111$ satisfies clauses 1, 2 and 5 whatever d, e, f are. P_1 has a good “tail”: $def = 110$ satisfies clauses 3 and 4 whatever a, b, c are. Put together in C_1 , clause 6 $(a \vee \neg d \vee f)$ is also satisfied, by $a = 1$ from the head. Crossover can combine good *building blocks* from different parents in one step — something no single-bit local move can do.

3.5 How a population moves on the landscape



The initial population may be randomly distributed, but as the GA runs the population has more members around the peaks — and especially around the highest peaks, which reproduce most. Unlike iterated hill climbing, members also *share information* through crossover.

3.6 A tiny example (Goldberg, 1989)

Chromosome: a 5-bit string x . Fitness $f(x) = x^2$ (the square of the binary number). Population of 4. Single-point crossover.

Initial	x	$f(x)$	$f / \sum f$	expected = $f / \bar{f}$	actual	selected
01101	13	169	0.14	0.58	1	01101
11000	24	576	0.49	1.97	2	11000
01000	8	64	0.06	0.22	0	11000
10011	19	361	0.31	1.23	1	10011
1170			avg 292.5			

Crossover: pair (0110|1, 1100|0) cut after bit 4; pair (11|000, 10|011) cut after bit 2:

Selected	offspring	x	$f(x)$
01101	01100	12	144
11000	11001	25	625
11000	11011	27	729
10011	10000	16	256
			1754 (avg 438.5)

A fitter population: average fitness rose from 292.5 to 438.5 and the best from 576 to 729.

Cycle 2: probabilities 0.08, 0.36, 0.42, 0.16; expected copies 0.33, 1.43, 1.66, 0.58; the slide's draw selects 11001, 11001, 11011, 11011: total 2708, average 677.

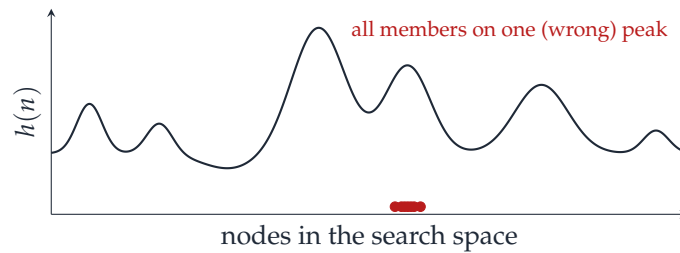
Slide erratum: Cycle 2 average

The cycle 2 slide lists "Total 1754, Avg. 493" for the population before selection. $1754/4 = 438.5$, as on the previous slide ("Avg. 439"); 493 is a digit transposition.

Exam trap: A fitter population, but less diverse

After cycle 2 every member has 11___ and the middle bit is 0 in all four (11001, 11011). The optimum 11111 ($f = 961$) needs a 1 in the middle, and **no crossover can create it**: crossover only recombines bits present in the population. Only **mutation** can restore the missing gene.

Selection pressure causes *premature convergence*; this is why “a large, diverse population is critical for performance”.



The population may become less diversified: it can converge on a local peak.

3.7 Practical matters

- Candidates are encoded as **chromosomes**; **genes** are like components of the candidate.
- GAs work by inheriting components from two parents, so a **careful encoding** is needed: crossover must produce valid candidates that inherit meaningful pieces (TSP next).
- A **large, diverse population** is critical for performance. Typical mutation rates are low (e.g. about one bit per chromosome or less per generation).

Module recap

- Evolution as generate-and-test: variation by recombination, selection by fitness.
- GA loop: fitness-proportional selection (roulette wheel), crossover, rare mutation, replacement.
- Expected copies of a member = $f_i / \bar{f}$.
- Crossover can join good building blocks from two parents (SAT example: two parents with $h = 5, 4$ give a child with $h = 6$).
- Selection reduces diversity; mutation reintroduces missing genes.

Module 4 Genetic Algorithms for the TSP

4.1 Setting up the GA

- Create a population of candidate tours. The objective is the **cost** of the tour: a **minimisation** problem, so the cheapest tours are the fittest (use e.g. fitness = 1/cost for roulette selection).
- **Path representation**: a tour is a permutation of the cities, with the return from the last city to the first understood.
- **Selection**: clone each tour in proportion to fitness. **Crossover**: randomly pair and cross. **Mutation**: randomly permute a tour once in a while (e.g. swap two cities, or reverse a segment).

Throughout this module, with 15 cities A–O:

$$P_1 = \text{O D G L A H K M B J F C N I E}, \quad P_2 = \text{H G M F O A D K I C N E L B J}.$$

(Every child below was recomputed by code and matches the slides.)

4.2 Single-point crossover does not work

Cut both after position 10:

P_1	O	D	G	L	A	H	K	M	B	J	F	C	N	I	E
P_2	H	G	M	F	O	A	D	K	I	C	N	E	L	B	J
C_1	O	D	G	L	A	H	K	M	B	J	N	E	L	B	J
C_2	H	G	M	F	O	A	D	K	I	C	F	C	N	I	E

Red cells are repeats: C_1 visits L, B and J twice and never visits F, C, I; C_2 visits F, C and I twice and misses L, B, J. **Both offspring are not valid tours.** A permutation encoding needs crossover operators that preserve “each city exactly once”.

4.3 Cycle Crossover (CX)

Formula / Derivation: Step 1: identify cycles

Start at position 1: P_1 has O. Find O in P_1 's column partner: P_2 at position 1 is H; H is at position 6 of P_1 ; P_2 at position 6 is A; A is at position 5 of P_1 ; P_2 at position 5 is O — back to the start. So positions {1, 5, 6} (cities O, A, H) form **cycle 1**: within those positions both parents contain the same set of cities.

Repeat from the first unassigned position. Cycle labels for positions 1–15 (computed):

P_1	O	D	G	L	A	H	K	M	B	J	F	C	N	I	E
cycle	1	2	2	3	1	1	2	2	4	5	3	5	3	4	5
P_2	H	G	M	F	O	A	D	K	I	C	N	E	L	B	J

Cycle 1 = positions {1, 5, 6} (O, A, H); cycle 2 = {2, 3, 7, 8} (D, G, K, M); cycle 3 = {4, 11, 13} (L, F, N); cycle 4 = {9, 14} (B, I); cycle 5 = {10, 12, 15} (J, C, E).

Formula / Derivation: Step 2: build the children

C_1 takes **odd**-numbered cycles from P_1 and **even**-numbered cycles from P_2 ; C_2 the opposite:

C_1	O	G	M	L	A	H	D	K	I	J	F	C	N	B	E
C_2	H	D	G	F	O	A	K	M	B	C	N	E	L	I	J

Because every cycle occupies the same set of positions and contains the same cities in both parents, taking whole cycles from either parent always yields a valid permutation. Each city also keeps the **absolute position** it had in one of the parents.

4.4 Partially Mapped Crossover (PMX)**Formula / Derivation: PMX, step by step**

Choose a subtour: positions 6–10. Copy it from P_1 into C_1 and record the mapping between the parents in those positions:

$$H \leftrightarrow A, \quad K \leftrightarrow D, \quad M \leftrightarrow K, \quad B \leftrightarrow I, \quad J \leftrightarrow C.$$

Now copy the remaining positions from P_2 . Where P_2 's city is already in the copied subtour, follow the map until reaching a city that is not:

- Position 1: P_2 has H, already used; $H \rightarrow A$; A is free: place A.
- Position 3: P_2 has M; $M \rightarrow K$; K is also used ("remember that K is already in C_1 "); $K \rightarrow D$: place D.
- Position 14: P_2 has B; $B \rightarrow I$: place I. Position 15: $J \rightarrow C$: place C.
- All other positions take P_2 's city directly.

P_1	O	D	G	L	A	H	K	M	B	J	F	C	N	I	E
P_2	H	G	M	F	O	A	D	K	I	C	N	E	L	B	J
C_1	A	G	D	F	O	H	K	M	B	J	N	E	L	I	C
C_2	O	M	G	L	H	A	D	K	I	C	F	J	N	B	E

C_2 is built the same way, first copying the subtour A D K I C from P_2 . Red cells were placed by following the map.

Intuition: What each child inherits

C_1 keeps a *contiguous piece of tour* (H–K–M–B–J, i.e. four actual edges) from P_1 , and as far as possible the *positions* of the other cities from P_2 . For TSP, contiguous subtours are what matter — they are the edges.

4.5 Order Crossover (OX)**Formula / Derivation: Order crossover**

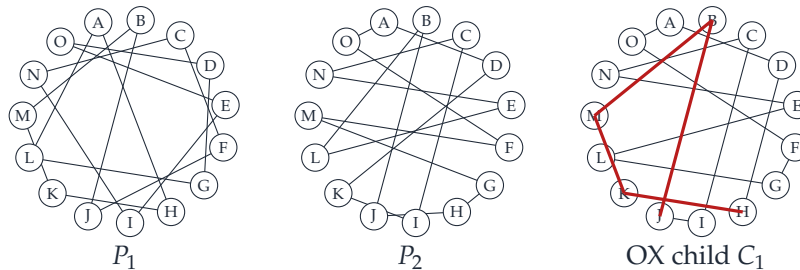
Copy a subtour from P_1 into C_1 (positions 6–10: H K M B J). Fill the remaining positions with the other cities **in the order they occur in** P_2 : from H G M F O A D K I C N E L B J delete H, K, M, B, J to get G F O A D I C N E L, then fill positions 1–5 and 11–15 in that order:

C_1	G	F	O	A	D	H	K	M	B	J	I	C	N	E	L
C_2	O	G	L	H	M	A	D	K	I	C	B	J	F	N	E

C_2 : copy A D K I C from P_2 , then O, G, L, H, M, B, J, F, N, E from P_1 in order.

Note: Where to start filling

The slides fill from position 1. Davis's original OX fills starting just *after* the copied segment and wraps around; the resulting permutation is a rotation of a different fill but preserves the same relative order. If a question specifies a convention, follow it.



Tours drawn schematically on a circle (as the slides say, real cities are not laid out in a circle). Red: the four edges of the copied subtour.

4.6 Adjacency representation

Definition: Adjacency representation

Position i (for city i in alphabetical order, the INDEX) holds the city that comes **next** after city i in the tour.

INDEX	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
P_1 (adj.)	H	J	N	G	O	C	L	K	E	F	M	A	B	I	D
P_2 (adj.)	D	J	N	K	L	O	M	G	C	H	I	B	F	E	A

From $P_1 = O D G L A H \dots$: A is followed by H, so position A holds H; B is followed by J; $\dots$; O is followed by D.

Formula / Derivation: Not every permutation is a tour

B A N G O C L K E F M J H I D says $A \rightarrow B$ and $B \rightarrow A$: a 2-city cycle, not a tour. A permutation is a valid adjacency tour iff, read as a function, it is a **single cycle** through all n cities.

How many representations does a tour have? In path representation a tour of n cities has $2n$ representations (n starting cities $\times$ 2 directions). In adjacency representation there is no starting city: a directed tour has exactly **one** representation, so an undirected tour has **two** (one per direction). So the adjacency representation can represent every tour, but not every permutation is a tour.

4.6.1 Crossover operators for adjacency representation

- **Alternating edges**: from a city X take the next city Y from P_1 ; from Y take the next from P_2 ; and so on.
- **Heuristic crossover**: for each city choose the next city from whichever parent gives the *closer* (shorter) edge.

The adjacency representation facilitates these choices: “the edge out of city i ” is just position i .

Formula / Derivation: Alternating edges — careful

Start at F. From P_1 : $F \rightarrow C$. From P_2 : $C \rightarrow N$. From P_1 : $N \rightarrow I$. From P_2 : $I \rightarrow C$ — but C is already in the child. **Oops!** The edge would close a premature cycle $F-C-N-I-C$. Rule: when the chosen edge would revisit a city, choose a random (or nearest) unvisited next city instead.

4.7 Ordinal representation**Formula / Derivation: Encoding a tour**

Keep an ordered reference list of the cities not yet used (initially A, B, ..., O with indices 1–15). For each city of the tour in turn, write its **current index** in the list, then **remove it** from the list (all later cities move up by one).

City	index	remaining list after removal
O	15	A B C D E F G H I J K L M N
D	4	A B C E F G H I J K L M N
G	6	A B C E F H I J K L M N
L	10	A B C E F H I J K M N
A	1	B C E F H I J K M N
...	and so on	

$P_1 \mapsto (15, 4, 6, 10, 1, 5, 7, 7, 1, 5, 3, 1, 3, 2, 1)$ and $P_2 \mapsto (8, 7, 11, 6, 11, 1, 3, 6, 4, 2, 5, 2, 3, 1, 1)$ (computed; P_1 matches the slide).

Decoding reverses the process: repeatedly take the city at the given index from the current list.

Formula / Derivation: Why single-point crossover now works

The i -th entry of an ordinal string can be any integer from 1 to $n - i + 1$ (the size of the remaining list), and *every* such string decodes to a valid tour. Any mix of two ordinal strings position by position keeps each entry in its legal range, so **single-point crossover always produces valid offspring**. Try it (cut after position 7):

$$C_1 = (15, 4, 6, 10, 1, 5, 7 \mid 6, 4, 2, 5, 2, 3, 1, 1) \mapsto \text{O D G L A H K J F C N E M B I}$$

$$C_2 = (8, 7, 11, 6, 11, 1, 3 \mid 7, 1, 5, 3, 1, 3, 2, 1) \mapsto \text{H G M F O A D L B K I C N J E}$$

Both are permutations of A–O. The drawback: the meaning of an entry depends on everything before it, so the tail inherited from a parent does not correspond to that parent's actual tour edges — crossover here preserves little useful structure.

Operator	What the child inherits	Valid by construction?
Single-point (path)	two chunks of positions	no
Cycle (CX)	absolute positions of cities	yes
PMX	a subtour from one parent, positions from the other	yes
Order (OX)	a subtour from one parent, relative order from the other	yes
Alternating edges (adjacency)	edges alternately from both parents	needs repair
Heuristic (adjacency)	the shorter outgoing edge at each city	needs repair
Single-point (ordinal)	encoded choices	yes, but weak inheritance

Module recap

- Path representation + single-point crossover gives invalid tours.
- CX: identify position cycles; take odd cycles from P_1 , even from P_2 .
- PMX: copy a subtour; place the rest from P_2 , following the mapping on conflicts.
- OX: copy a subtour; fill the rest in P_2 's order.
- Adjacency: position i holds the successor of city i ; 2 representations per tour; alternating-edges and heuristic crossover (with repair).
- Ordinal: index in a shrinking list; single-point crossover always valid.

Module 5 Emergent Systems: Complex Behaviour from Simple Elements

Before the last algorithm (ant colonies), the lecture steps back to the idea that makes population methods work: simple parts, interacting, can produce sophisticated wholes.

Definition: Emergent system

A collection of simple entities that organise themselves so that a larger, more sophisticated entity emerges. The behaviour of the complex system is a property that **emerges from interactions amongst its components**, not from any one component.

Examples from the slides:

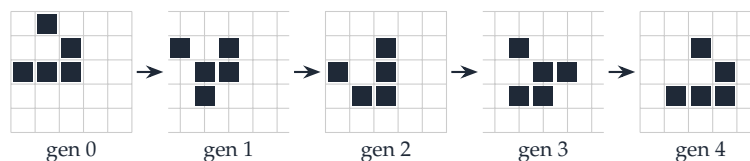
- an **ant colony** behaves like a living organism that can find food very quickly;
- the **brain** is made of billions of comparatively simple neurons;
- a **flock of birds** moves in synchrony; **termite mounds** are elaborate structures;
- **stock market** behaviour, formation of **sand dunes**, **hurricanes and cyclones**.

5.1 Conway's Game of Life (1970)

Definition: Game of Life

A **cellular automaton**: an infinite grid of cells, each alive or dead. At each time step every cell looks at its 8 neighbours and updates simultaneously:

Cell state	alive neighbours	new state	explanation
alive	< 2	dead	lonely
alive	2 or 3	alive	stable
alive	> 3	dead	overcrowded
dead	exactly 3	alive	resurrection
dead	otherwise	dead	—



A glider (computed by applying the rules). After 4 generations the same shape reappears one cell down and one cell right: an *illusion of movement*. No cell moves; the pattern does.

Formula / Derivation: Checking one step: the blinker

Three live cells in a row, $(0, 1)$, $(1, 1)$, $(2, 1)$. The centre $(1, 1)$ has 2 live neighbours: survives. The ends each have 1: die. Cells $(1, 0)$ and $(1, 2)$ are dead with exactly 3 live neighbours: born. Result: a vertical line $(1, 0)$, $(1, 1)$, $(1, 2)$. One more step gives back the horizontal line — a period-2 oscillator.

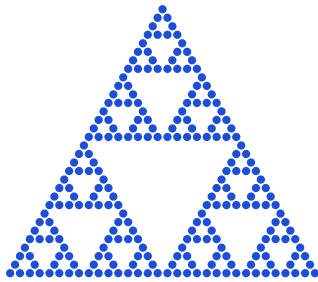
Note: Gosper's glider gun

The slide shows Gosper's glider gun, a finite pattern that emits a new glider every 30 generations, forever. Such patterns show that the Game of Life can simulate logic gates and is in fact **Turing-complete** — four rules on a grid can compute anything a computer can.

5.2 Chaos and fractals

Definition: Fractal

A never-ending pattern: infinitely complex, **self-similar across different scales**, created by repeating a simple process over and over in a feedback loop. Driven by recursion, fractals are images of dynamic systems — the pictures of chaos. Nature is full of them: trees, rivers, coastlines, mountains, clouds, seashells, hurricanes (Fractal Foundation).



Sierpinski triangle (Wacław Sierpiński, 1915). Each filled triangle contains three half-size copies of the whole.

The figure is drawn by a surprisingly simple rule: plot a dot at row n , position k whenever the binomial coefficient $\binom{n}{k}$ is *odd* (Pascal's triangle mod 2), for $n < 32$.

Formula / Derivation: Fractal dimension of the Sierpinski triangle

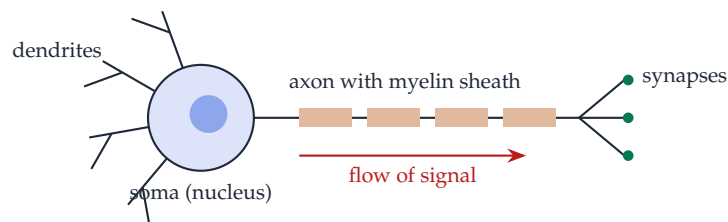
Scaling the triangle by a factor $\frac{1}{2}$ gives 3 copies of itself. A set with N copies at scale s has similarity dimension

$$D = \frac{\log N}{\log(1/s)} = \frac{\log 3}{\log 2} \approx 1.585.$$

It is “more than a line, less than an area”. After k subdivision steps the filled area is $(3/4)^k$ of the original, which tends to 0.

5.3 The brain

- “The most complex object in the known universe” (Christof Koch, Allen Institute for Brain Science). About 1.4 kg, roughly **100 billion neurons**, each contacting thousands or tens of thousands of others.
- About 10^{11} neurons $\times 10^4$ connections $\approx 10^{15}$, i.e. some 100 trillion connections (the slide's newspaper quote gives 100 trillion). Laid end to end, the neurons would wrap round the Earth twice (*The Independent*, 2014).



A biological neuron receives inputs through its **dendrites** and sends a signal down its **axon**; the axon branches and connects to other neurons' dendrites via **synapses**, which transmit the signal chemically.

Definition: Computational neuron and ANN

A computational neuron is a simple device computing some function $y = f(x_1, \dots, x_n)$ of its inputs. A **feedforward artificial neural network** (input, hidden and output layers) encapsulates a function from input to output. It learns the function by **training**: many labelled patterns are shown, the network produces an output, and **backpropagation** sends the error back to tune the connection weights (Week 1 notes derive it).

5.4 Karl Sims: evolved virtual creatures (1994)

Sims evolved simulated creatures with a genetic algorithm. The **genotype** is a directed graph (it can be recursive, so one gene can produce repeated body parts); the **phenotype** is a hierarchy of 3-D parts with sensors and muscles, simulated with physics. With populations of hundreds and a fitness task such as locomotion, different walking, swimming and jumping designs evolved — a vivid demonstration of genotype/phenotype and of GAs discovering designs nobody wrote down. (See also the documentary *The Secret Life of Chaos*.)

Module recap

- Emergence: complex behaviour arising from interactions of simple parts (ants, brains, flocks, markets, weather).
- Game of Life: four local rules; gliders show emergent “movement”; the system is Turing-complete.
- Fractals: self-similarity from recursion; Sierpinski triangle has dimension $\log 3 / \log 2 \approx 1.585$.
- Brain: $\sim 10^{11}$ neurons and $\sim 10^{14}$ – 10^{15} connections; ANNs abstract neurons as functions and learn by backprop.
- Karl Sims: GAs evolving creatures’ bodies and brains.

Module 6 Ant Colony Optimization

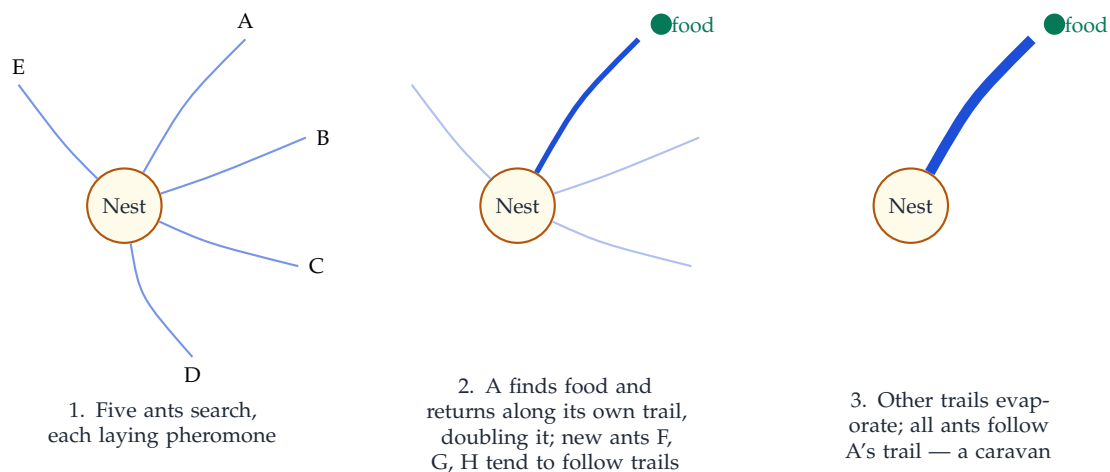
6.1 Biosemiotics: communicating through signs

Definition: Semiotics and biosemiotics

Semiotics: the study of signs and symbols and their use or interpretation (road signs, for example). **Biosemiotics** (Greek *bios*, life, and *sēmeiōtikos*, observant of signs) studies pre-linguistic meaning-making: the production and interpretation of signs and codes in the biological realm. Animals frequently mark their territory; ants mark trails.

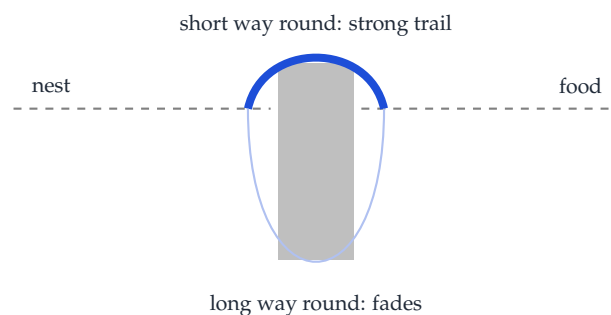
6.2 Ant colonies and foraging

Ants live in complex social colonies: a queen, and workers who forage for food and protect the nest. “Ant colony” describes both the physical structure and the social rules by which ants organise their work. Some workers become foragers; cooperation, division of labour and well-developed communication let ants exploit their environment in ways few other animals approach.



Intuition: Positive feedback plus evaporation

More ants on a trail $\Rightarrow$ more pheromone $\Rightarrow$ more ants choose it (**positive feedback**). Unused trails fade (**evaporation**), so the colony forgets bad choices. When an obstacle is placed on a trail, ants going round the shorter side complete trips faster, so that side accumulates pheromone faster: the colony **quickly finds the shortest diversion** — with no ant knowing the geometry.



6.3 Ant Colony Optimization (ACO)

Definition: ACO

An optimisation method inspired by the biosemiotic communication between ants (Marco Dorigo, 1992). Each ant **constructs a solution** with a **stochastic greedy method** that combines a **heuristic function** with **pheromone-trail following**. Related to **swarm optimisation**; applicable to problems that can be reduced to **graph search**. Illustrated here with the TSP.

Algorithm: TSP-ACO() — from the slide (more detailed version)

```

1: bestTour ← nil
2: repeat
3:   randomly place M ants on N cities
4:   for each ant a do                                     ▷ construct tour
5:     for n ← 1 to N do
6:       ant a selects an edge from the distribution  $P_n^a$ 
7:     end for
8:   end for
9:   update bestTour
10:  for each ant a do                                     ▷ update pheromone
11:    for each edge  $(u, v)$  in the ant's tour do
12:      deposit pheromone  $\propto 1/\text{tour-length}$  on edge  $(u, v)$ 
13:    end for
14:  end for
15: until some termination criteria
16: return bestTour

```

6.4 The transition rule

Formula / Derivation: Probability that ant *k* moves from city *i* to city *j*

$$P_{ij}^k(t) = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha [\eta_{ij}]^\beta}{\sum_{h \in \text{allowed}_k(t)} [\tau_{ih}(t)]^\alpha [\eta_{ih}]^\beta} & \text{if } j \in \text{allowed}_k(t), \\ 0 & \text{otherwise,} \end{cases}$$

where

- $\tau_{ij}(t)$ = pheromone on edge *ij* (learned, shared memory of the colony);
- η_{ij} = **visibility**, inversely proportional to distance, typically $\eta_{ij} = 1/d_{ij}$ (the greedy heuristic);
- $\text{allowed}_k(t)$ = cities ant *k* has not yet visited (so every ant builds a valid tour);
- $\alpha, \beta \geq 0$ weight pheromone vs visibility.

The denominator normalises so that the probabilities over allowed cities sum to 1.

Formula / Derivation: Worked example

An ant at A may go to B, C or D, with $d_{AB} = 2$, $d_{AC} = 4$, $d_{AD} = 5$ and pheromone $\tau_{AB} = 1$, $\tau_{AC} = 2$, $\tau_{AD} = 1$. Weights $w_j = \tau_{Aj}^\alpha (1/d_{Aj})^\beta$ (computed):

Setting	w_B	w_C	w_D	$P(B), P(C), P(D)$
$\alpha = 1, \beta = 1$	0.500	0.500	0.200	0.417, 0.417, 0.167
$\alpha = 1, \beta = 2$	0.250	0.125	0.040	0.602, 0.301, 0.096
$\alpha = 0, \beta = 2$ (distance only)	0.250	0.0625	0.040	0.709, 0.177, 0.113
$\alpha = 1, \beta = 0$ (pheromone only)	1	2	1	0.25, 0.50, 0.25

Row 2 in detail: $\text{sum} = 0.25 + 0.125 + 0.04 = 0.415$; $P(B) = 0.25/0.415 = 0.602$.

With $\alpha = 0$ the ants are a *randomised nearest-neighbour* heuristic; with $\beta = 0$ they ignore distance and only follow the colony's memory. Both extremes are poor; typical settings are $\alpha \approx 1$, $\beta \approx 2$ –5.

6.5 Updating pheromone

Formula / Derivation: Deposit and evaporation

After constructing a tour in n time steps, each ant k deposits Q/L_k on every edge it traversed, where L_k is the length of its tour and Q is a constant — **shorter tours deposit more**. The total deposit on edge ij is

$$\Delta\tau_{ij}(t, t+n) = \sum_{k: ij \in \text{tour}_k} \frac{Q}{L_k},$$

and the pheromone is updated as

$$\tau_{ij}(t+n) = (1-\rho)\tau_{ij}(t) + \Delta\tau_{ij}(t, t+n),$$

where $\rho \in (0, 1)$ is the **rate of evaporation**.

Example ($\rho = 0.5$, $Q = 10$): edge AB had $\tau = 1$ and was used by two ants with tours of length 20 and 25: $\tau \leftarrow 0.5(1) + \frac{10}{20} + \frac{10}{25} = 0.5 + 0.5 + 0.4 = \mathbf{1.4}$. Edge AC had $\tau = 2$, used by the ant with tour 25: $1 + 0.4 = \mathbf{1.4}$. Edge AD had $\tau = 1$, unused: $\mathbf{0.5}$.

Formula / Derivation: Why evaporation matters

Without deposits, τ decays geometrically: $\tau(t+kn) = (1-\rho)^k\tau(t)$; after $k = 10$ tours with $\rho = 0.1$, only $0.9^{10} \approx 35\%$ remains. If an edge receives a constant deposit Δ every round, τ converges to the fixed point $\tau^* = (1-\rho)\tau^* + \Delta \Rightarrow \tau^* = \Delta/\rho$. Evaporation therefore bounds pheromone, lets the colony *forget* early poor choices, and prevents premature convergence on the first good tour found.

	Simulated Annealing	Genetic Algorithm	Ant Colony Optimization
Candidates	one current node	a population	a colony of ants building solutions
Randomness	which neighbour; whether to accept	selection, crossover point, mutation	next city at each step
Memory	current node, bestN-ode	the population (genes)	pheromone on edges
Exploration control	temperature T	mutation, population diversity	α, β , evaporation ρ
Builds or perturbs?	perturbs	recombines	constructs (stochastic greedy)

Module recap

- Ants communicate via pheromone (biosemiotics); positive feedback and evaporation find short paths.
- ACO (Dorigo 1992): ants build tours stochastically with $P_{ij} \propto \tau_{ij}^\alpha \eta_{ij}^\beta$ over unvisited cities.
- Pheromone update: $\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_k Q/L_k$ over ants that used ij .
- All three randomised methods trade off exploration and exploitation with tunable parameters.

Practice: Test Yourself

Work each problem before reading its solution. All numerical answers were checked by running code.

Part A — Numerical problems and traces

Problem N1

[Module 2] Sigmoid acceptance. Compute $P = 1/(1 + e^{-\Delta E/T})$ for (a) $\Delta E = 4, T = 2$; (b) $\Delta E = -4, T = 2$; (c) $\Delta E = -4, T = 8$; (d) $\Delta E = -10, T = 2$. What do (b) and (c) show?

Solution

(a) $1/(1 + e^{-2}) = 1/1.135 = \mathbf{0.881}$. (b) $1/(1 + e^2) = 1/8.389 = \mathbf{0.119}$ — note (a)+(b) = 1 (symmetry). (c) $1/(1 + e^{0.5}) = 1/2.649 = \mathbf{0.378}$. (d) $1/(1 + e^5) = \mathbf{0.007}$.
(b) vs (c): the same bad move is accepted three times as often at the higher temperature. (b) vs (d): at a fixed temperature, a much worse move is almost never accepted.

Problem N2

[Module 2] Effect of ΔE . The current node has $eval = 50$ and $T = 5$. Compute the probability of moving to neighbours with $eval = 40, 48, 50, 55, 65$.

Solution

$P = 1/(1 + e^{-(eval(v_n) - 50)/5})$: $40 \rightarrow \mathbf{0.119}$, $48 \rightarrow \mathbf{0.401}$, $50 \rightarrow \mathbf{0.500}$, $55 \rightarrow \mathbf{0.731}$, $65 \rightarrow \mathbf{0.953}$.
E.g. for 48: $\Delta E = -2$, $e^{0.4} = 1.492$, $P = 1/2.492 = 0.401$.

Problem N3

[Module 2] Simulated annealing limits. Show that as $T \rightarrow \infty$ the sigmoid rule accepts every move with probability $\frac{1}{2}$, and as $T \rightarrow 0^+$ it becomes Hill Climbing on randomly chosen neighbours.

Solution

$T \rightarrow \infty$: $\Delta E/T \rightarrow 0$, $e^{-\Delta E/T} \rightarrow 1$, $P \rightarrow \frac{1}{2}$ for every move: a coin-flip random walk.
 $T \rightarrow 0^+$: if $\Delta E > 0$, $-\Delta E/T \rightarrow -\infty$, $e^{-\Delta E/T} \rightarrow 0$, $P \rightarrow 1$ (always accept improvements); if $\Delta E < 0$, $e^{-\Delta E/T} \rightarrow \infty$, $P \rightarrow 0$ (never accept worse). If $\Delta E = 0$, $P = \frac{1}{2}$ at every T . So at $T \approx 0$ the algorithm accepts only improvements: stochastic (first-improvement) Hill Climbing.

Problem N4

[Module 3] Roulette-wheel selection. Population 10110, 01011, 11100, 00111 with $f(x) = x^2$. Compute fitness, selection probabilities, expected copies, and the cumulative probabilities used to implement the wheel. Which member is selected by random draws 0.30, 0.40, 0.70, 0.98?

Solution

$x = 22, 11, 28, 7$; $f = 484, 121, 784, 49$; total 1438, average 359.5.
 Probabilities 0.337, 0.084, 0.545, 0.034; expected copies $f/\bar{f} = 1.35, 0.34, 2.18, 0.14$.
 Cumulative: 0.337, 0.421, 0.966, 1.000.
 Draws: 0.30 $\rightarrow$ 10110; 0.40 $\rightarrow$ 01011; 0.70 $\rightarrow$ 11100; 0.98 $\rightarrow$ 00111.

Problem N5

[Module 3] Crossover and the missing gene. Cross 11100 with 01011 after bit 3, and 11100 with 10110 after bit 2. What are the children and their fitness under $f = x^2$? Could any crossover in the population of Problem N4 produce 11111?

Solution

111|00 $\times$ 010|11 $\Rightarrow$ 11111 ($31^2 = 961$, the optimum) and 01000 (64).
 11|100 $\times$ 10|110 $\Rightarrow$ 11110 (900) and 10100 (400).
 Yes: every bit position has a 1 somewhere in the population (e.g. positions 1–3 in 11100, positions 4–5 in 01011), so crossover can assemble 11111. Contrast Goldberg's cycle 2, where the middle bit was 0 in every member and only mutation could help.

Problem N6

[Module 3] SAT fitness. For $F = (a \vee \neg b \vee e) \wedge (\neg a \vee b \vee c) \wedge (\neg c \vee d \vee f) \wedge (d \vee \neg e \vee f) \wedge (\neg a \vee b \vee d) \wedge (a \vee \neg d \vee f)$ (bits $abcdef$), evaluate 101001 and 010110, cross them after bit 3, and evaluate both children.

Solution

101001: clauses 1, 1, 1, 1, 0, 1: $h = 5$ (clause 5 $\neg a \vee b \vee d$ fails: $a = 1, b = 0, d = 0$).
 010110: 1, 1, 1, 1, 1, 0: $h = 5$ (clause 6 $a \vee \neg d \vee f$ fails).
 Children: 101|110 = 101110: $h = 6$ (solution); 010|001 = 010001: clauses 0, 1, 1, 1, 1, 1 — clause 1 $a \vee \neg b \vee e$ fails ($a = 0, b = 1, e = 0$): $h = 5$. The head $abc = 101$ with the tail $def = 110$ satisfies everything.

Problem N7

[Module 4] Cycle crossover. $P_1 = A B C D E F G H$, $P_2 = C F A E B H D G$. Find the cycles and both children.

Solution

Position 1: P_1 A; P_2 has C there; C is at P_1 position 3; P_2 position 3 is A — back to A. Cycle 1 = positions {1, 3}. Position 2: B; P_2 F $\rightarrow$ P_1 position 6; P_2 H $\rightarrow$ 8; P_2 G $\rightarrow$ 7; P_2 D $\rightarrow$ 4; P_2 E $\rightarrow$ 5; P_2 B $\rightarrow$ back. Cycle 2 = {2, 4, 5, 6, 7, 8}.
 C_1 (odd cycles from P_1 , even from P_2) = A F C E B H D G. C_2 = C B A D E F G H.

Problem N8

[Module 4] PMX and OX. Same parents; copy positions 3–5 from the first parent. Give C_1 by PMX and by OX.

Solution

Segment from P_1 : C D E. Mapping $C \leftrightarrow A$, $D \leftrightarrow E$, $E \leftrightarrow B$.

PMX: position 1: P_2 has C (used) $\rightarrow$ A: **A**. Position 2: F. Position 6: H. Position 7: P_2 has D (used) $\rightarrow$ E (used) $\rightarrow$ B: **B**. Position 8: G. $C_1 = A F C D E H B G$.

OX: remaining cities in P_2 order: F A B H G; fill positions 1–2 then 6–8: $C_1 = F A C D E B H G$.

Problem N9

[Module 4] Adjacency and ordinal representations. Encode $P_2 = C F A E B H D G$ in adjacency and ordinal representations. Decode the ordinal string $(3, 1, 4, 1, 2, 2, 1, 1)$.

Solution

Adjacency (successor of A, B, ..., H): $A \rightarrow E$, $B \rightarrow H$, $C \rightarrow F$, $D \rightarrow G$, $E \rightarrow B$, $F \rightarrow A$, $G \rightarrow C$, $H \rightarrow D$: E H F G B A C D.

Ordinal: C is 3rd of ABCDEFGH (list ABDEFGH); F is 5th (ABDEGH); A 1st (BDEGH); E 3rd (BDGH); B 1st (DGH); H 3rd (DG); D 1st (G); G 1st: $(3, 5, 1, 3, 1, 3, 1, 1)$.

Decode $(3, 1, 4, 1, 2, 2, 1, 1)$: from ABCDEFGH take 3rd: C; from ABDEFGH 1st: A; from BDEFGH 4th: F; from BDEGH 1st: B; from DEGH 2nd: E; from DGH 2nd: G; D; H. Tour C A F B E G D H.

Problem N10

[Module 6] ACO transition probabilities. An ant at city A can go to B, C, D with $d = 3, 1, 2$ and $\tau = 2, 1, 1$. With $\alpha = 1$, $\beta = 2$, $\eta = 1/d$, compute the probabilities.

Solution

$w_B = 2 \cdot (1/3)^2 = 0.222$, $w_C = 1 \cdot 1^2 = 1$, $w_D = 1 \cdot (1/2)^2 = 0.25$; sum 1.472.

$P(B) = 0.151$, $P(C) = 0.679$, $P(D) = 0.170$. B has twice the pheromone of C, but C is three times closer and visibility is squared, so C wins.

Problem N11

[Module 6] Pheromone update. $\rho = 0.2$, $Q = 20$. Edge e has $\tau = 3$. (a) It is used by one ant whose tour has length 40. (b) It is used by two ants with tours of length 40 and 50. (c) It is not used. Find the new τ in each case.

Solution

Evaporation gives $0.8 \times 3 = 2.4$. (a) $2.4 + 20/40 = 2.9$. (b) $2.4 + 0.5 + 0.4 = 3.3$. (c) 2.4. Long-run level for a constant deposit Δ per round: $\tau^* = \Delta/\rho$; for (a), $0.5/0.2 = 2.5$.

Problem N12

[Module 1] Random restarts. The global maximum's basin covers 20% of the space. How many random restarts of Hill Climbing give at least a 95% chance of success?

Solution

Need $0.8^k \leq 0.05$: $k \geq \ln 0.05 / \ln 0.8 = 13.4$, so **14** restarts.

Problem N13

[Module 5] Fractal dimension. The Sierpinski *carpet* is made by dividing a square into 3×3 and removing the centre, recursively. Compute its dimension and compare with the triangle.

Solution

8 copies at scale $\frac{1}{3}$: $D = \log 8 / \log 3 = \mathbf{1.893}$. The triangle (3 copies at scale $\frac{1}{2}$) has 1.585. The carpet is “closer to an area” because it removes a smaller fraction at each step ($\frac{1}{9}$ vs $\frac{1}{4}$).

Problem N14

[Module 4] Counting representations. For 8 cities, how many path-representation strings describe the same tour? How many adjacency strings? How many distinct tours are there?

Solution

Path: $2n = \mathbf{16}$ (8 starting cities $\times$ 2 directions). Adjacency: $\mathbf{2}$ (one per direction). Distinct tours: $(n-1)!/2 = 7!/2 = \mathbf{2520}$. Check: $8!/16 = 40320/16 = 2520$.

Part B — Subjective questions with model answers**Question S1**

Why are 2SAT and 3SAT so different in complexity, and what is the phase transition in random 3SAT?

Solution

A 2-literal clause $(x \vee y)$ equals the implications $\neg x \Rightarrow y$ and $\neg y \Rightarrow x$, so 2SAT reduces to reachability in an implication graph and is solvable in linear time (in P). A 3-literal clause does not force a single consequence when one literal is false, and 3SAT is NP-complete. For random 3SAT, the fraction of satisfiable formulas drops sharply from about 1 to about 0 around $m/n \approx 4.26$ clauses per variable; complete (deterministic) methods take longest near this ratio, because formulas are neither loose enough to have many solutions nor constrained enough to be refuted quickly.

Question S2

Contrast Random Walk, Hill Climbing and Stochastic Hill Climbing.

Solution

Random Walk moves to a random neighbour every time and remembers the best node seen: pure exploration, never stuck, but aimless. Hill Climbing moves only to a better neighbour and stops at a local optimum: pure exploitation of the gradient. Stochastic Hill Climbing picks a random neighbour and moves with probability $1/(1 + e^{-\Delta E/T})$: better moves are likely, worse moves possible but unlikely, blending exploration and exploitation, with T setting the balance.

Question S3

Explain Simulated Annealing: the physical analogy, the algorithm, and the role of temperature.

Solution

In annealing a metal is heated so atoms move freely, then cooled slowly so they settle into a low-energy crystal; cooling too fast (quenching) freezes defects. SA mirrors this: a candidate is the configuration, $-eval$ the energy, and T a parameter. Starting hot, it picks random neighbours and accepts them with sigmoid probability, so many downhill moves are accepted (exploration); after each epoch a cooling function lowers T , making it increasingly selective, until it behaves like Hill Climbing (exploitation). It keeps *bestNode*. Very large T gives a random walk ($P \approx \frac{1}{2}$); $T \rightarrow 0$ gives Hill Climbing.

Question S4

Why is SA more likely to move from a lower peak A to a higher peak B than back?

Solution

To go from A to B the search must descend into the valley between them, losing ΔE_A (the height of A above the valley); to go back it must lose $\Delta E_B > \Delta E_A$. Acceptance of a loss L scales roughly like $e^{-L/T}$, so the ratio of transition likelihoods is about $e^{(\Delta E_B - \Delta E_A)/T} > 1$. Over time the search spends more time on higher peaks, and as T falls the probability of leaving the highest peak vanishes. SA therefore suits jagged landscapes with an overall trend towards the optimum.

Question S5

Describe the components of a Genetic Algorithm and the role of each.

Solution

A population of N encoded candidates (chromosomes) is evaluated by a fitness function. **Selection** (roulette wheel) clones members in proportion to fitness, so fitter members get more copies (expected $f_i/\bar{f}$). **Crossover** pairs members randomly and recombines their genes (single-point, multi-point), combining good building blocks from different parents. **Mutation** occasionally flips genes, reintroducing diversity and genes missing from the whole population. **Replacement** puts the k strongest offspring in place of the k weakest members. The loop repeats until a termination criterion, returning the best member.

Question S6

Using Goldberg's example, explain how a GA improves a population and why diversity matters.

Solution

With $f(x) = x^2$ and population 01101, 11000, 01000, 10011 (average 292.5), selection gives 11000 two copies and 01000 none. Crossover produces 01100, 11001, 11011, 10000 (average 438.5, best 729). A second selection leaves 11001, 11001, 11011, 11011 (average 677). Fitness rose but diversity collapsed: every member now has 0 in the middle bit, so crossover can never reach the optimum 11111; only mutation can. Large, diverse populations and mutation guard against such premature convergence.

Question S7

Why does single-point crossover fail for TSP in path representation? Describe how CX, PMX and OX avoid the problem.

Solution

Cutting two permutations and swapping tails generally duplicates some cities and omits others, so children are not tours. CX partitions positions into cycles on which both parents hold the same set of cities and copies whole cycles alternately, preserving absolute positions. PMX copies a segment from one parent and fills the other positions from the second parent, resolving conflicts by following the mapping defined by the segment. OX copies a segment from one parent and fills the remaining positions with the missing cities in the order they appear in the other parent. All three yield valid permutations; PMX and OX preserve a contiguous subtour (actual edges), CX preserves positions.

Question S8

Explain the adjacency and ordinal representations, with one advantage and one drawback each.

Solution

Adjacency: position i holds the city visited after city i . It represents edges directly, which makes edge-based crossovers (alternating edges, heuristic crossover choosing the shorter outgoing edge) natural, and a tour has only two representations; but many permutations are not tours (they contain subcycles, e.g. $A \rightarrow B, B \rightarrow A$), so crossover needs repair. Ordinal: each city is replaced by its index in a shrinking list of unused cities; any string whose i -th entry lies in $1..n - i + 1$ is a valid tour, so single-point crossover always works; but an entry's meaning depends on everything before it, so offspring inherit little of their parents' actual tour structure.

Question S9

What is an emergent system? Illustrate with the Game of Life and ant colonies.

Solution

An emergent system is a collection of simple components whose interactions produce complex, organised behaviour not present in any component. In the Game of Life each cell follows four local rules about its neighbours, yet patterns such as gliders appear to move across the grid, and glider guns and logic circuits can be built — the system is Turing-complete. In an ant colony each ant follows simple rules (lay pheromone, prefer stronger trails), yet the colony as a whole finds short paths to food and routes around obstacles. ACO exploits exactly this emergent optimisation.

Question S10

Describe Ant Colony Optimization for TSP: tour construction, pheromone update, and the roles of α , β and ρ .

Solution

In each iteration M ants are placed randomly on cities. Each builds a tour step by step, moving from city i to an unvisited city j with probability proportional to $\tau_{ij}^\alpha \eta_{ij}^\beta$, where τ is pheromone (colony memory) and $\eta = 1/d$ is visibility (greedy heuristic). After all tours are built, the best tour is updated and every edge's pheromone evaporates by a factor $(1 - \rho)$ and receives Q/L_k from each ant k that used it, so edges in short tours are reinforced. α weights the colony's experience, β the distance heuristic ($\alpha = 0$ gives a randomised nearest-neighbour method), and ρ controls forgetting: too little evaporation causes premature convergence, too much prevents learning.

Question S11

Compare Simulated Annealing, Genetic Algorithms and ACO.

Solution

SA works with one candidate, perturbing it and accepting worse moves with a temperature-controlled probability; memory is just the current and best node. GAs work with a population, recombining candidates via crossover and occasionally mutating them; memory is the population's genes; exploration is controlled by diversity and mutation. ACO constructs new candidates each iteration with a stochastic greedy rule biased by pheromone; memory is the pheromone on edges; exploration is controlled by α , β and evaporation. All are stochastic local search methods that balance exploration and exploitation and return the best candidate found, with no optimality guarantee.

Part C — Multiple-choice questions

Q1. 2SAT is:

- (a) NP-complete (b) in P (c) undecidable (d) harder than 3SAT

Q2. Random 3SAT is hardest for complete methods around $m/n \approx$:

- (a) 2 (b) 3 (c) 4.3 (d) 8

Q3. A clause in CNF is satisfied when:

- (a) all literals are true (b) at least one literal is true (c) exactly one literal is true (d) no literal is negated

Q4. Random Walk is:

- (a) pure exploitation (b) pure exploration (c) Hill Climbing with restarts (d) tabu search

Q5. In stochastic Hill Climbing, a neighbour with $\Delta E = 0$ is accepted with probability:

- (a) 0 (b) 0.5 (c) 1 (d) depends on T

Q6. As $T \rightarrow \infty$ Simulated Annealing behaves like:

- (a) Hill Climbing (b) Random Walk (c) Beam Search (d) BFS

Q7. As $T \rightarrow 0$ Simulated Annealing behaves like:

- (a) Hill Climbing (b) Random Walk (c) a GA (d) DFS

Q8. With $eval(v_c) = 107$ and $T = 10$, the probability of moving to a neighbour with $eval = 100$ is about:

- (a) 0.06 (b) 0.33 (c) 0.50 (d) 0.78

- Q9.** For $\Delta E = +13$, P at $T = 10^{10}$ is about:
(a) 1.0 (b) 0.78 (c) 0.5 (d) 0
- Q10.** *CoolingFunction* in SA:
(a) raises T (b) lowers T after each epoch (c) resets T (d) sets $T = 0$
- Q11.** SA is more likely to move from lower peak A to higher peak B than the reverse because:
(a) B is closer (b) the loss needed to leave A is smaller (c) T increases (d) SA remembers B
- Q12.** SA works best on a landscape that is:
(a) a single smooth peak (b) jagged with a broad trend towards the optimum (c) a needle in a haystack (d) flat
- Q13.** “Things that persist, persist” is due to:
(a) Darwin (b) Dawkins (c) Steve Grand (d) Holland
- Q14.** Genetic Algorithms were devised by:
(a) Goldberg (b) John Holland (c) Dorigo (d) Conway
- Q15.** The phenotype is:
(a) the gene set (b) the physical expression of genes that competes (c) a crossover operator (d) a fitness function
- Q16.** In roulette-wheel selection the sector angle of a candidate is proportional to its:
(a) rank (b) fitness (c) age (d) length
- Q17.** Expected number of copies of a member in fitness-proportional selection is:
(a) f_i (b) $f_i / \bar{f}$ (c) $N f_i$ (d) $1 / f_i$
- Q18.** In Goldberg’s example, the expected copies of 11000 ($f = 576$, total 1170) are:
(a) 0.49 (b) 1.97 (c) 2.00 (d) 576
- Q19.** Mutation in a GA is mainly used to:
(a) speed up selection (b) reintroduce missing genes / diversity (c) sort the population (d) evaluate fitness
- Q20.** After cycle 2 in Goldberg’s example, which string can crossover alone never produce?
(a) 11001 (b) 11011 (c) 11111 (d) 11000
- Q21.** Single-point crossover of two TSP path-representation tours:
(a) always gives valid tours (b) can duplicate and omit cities (c) gives the optimal tour (d) is the same as PMX
- Q22.** Cycle crossover preserves:
(a) edges (b) absolute positions of cities (c) relative order only (d) nothing
- Q23.** In PMX, when a city from P_2 is already in the copied segment, the algorithm:
(a) skips it (b) follows the mapping until a free city is found (c) mutates (d) restarts
- Q24.** In Order Crossover, the cities outside the copied segment are placed:
(a) randomly (b) in the order they occur in the other parent (c) alphabetically (d) at their original positions
- Q25.** In the adjacency representation, position i holds:
(a) the i -th city visited (b) the city visited after city i (c) the index of city i (d) the distance to city i

- Q26.** How many adjacency representations does an undirected tour have?
 (a) 1 (b) 2 (c) n (d) $2n$
- Q27.** The adjacency string $B A \dots (A \rightarrow B, B \rightarrow A)$ is:
 (a) a valid tour (b) not a tour: it contains a 2-city cycle (c) the reverse tour (d) a mutation
- Q28.** In ordinal representation, single-point crossover:
 (a) always gives valid tours (b) never gives valid tours (c) needs repair (d) is undefined
- Q29.** The ordinal code of path $O D G L \dots$ (cities $A-O$) begins:
 (a) 15, 4, 6, 10 (b) 15, 4, 7, 12 (c) 1, 2, 3, 4 (d) 15, 3, 5, 9
- Q30.** In Conway's Game of Life, a dead cell with exactly 3 live neighbours:
 (a) stays dead (b) becomes alive (c) becomes overcrowded (d) is lonely
- Q31.** A live cell with 4 live neighbours:
 (a) survives (b) dies of overcrowding (c) dies of loneliness (d) splits
- Q32.** The similarity dimension of the Sierpinski triangle is:
 (a) 1 (b) 1.585 (c) 2 (d) 1.893
- Q33.** ACO was presented by:
 (a) Holland, 1975 (b) Marco Dorigo, 1992 (c) Karl Sims, 1994 (d) Conway, 1970
- Q34.** In the ACO transition rule, η_{ij} (visibility) is:
 (a) pheromone (b) proportional to distance (c) inversely proportional to distance (d) the evaporation rate
- Q35.** Setting $\alpha = 0$ in ACO makes the ants:
 (a) follow only pheromone (b) a randomised nearest-neighbour heuristic (c) random walkers (d) stop
- Q36.** An ant with tour length L_k deposits pheromone:
 (a) L_k (b) Q/L_k (c) QL_k (d) ρL_k
- Q37.** Pheromone update is $\tau_{ij} \leftarrow$:
 (a) $\rho\tau_{ij}$ (b) $(1 - \rho)\tau_{ij} + \Delta\tau_{ij}$ (c) $\tau_{ij} + \rho$ (d) $\Delta\tau_{ij}$ only
- Q38.** Evaporation in ACO helps to:
 (a) speed up tour construction (b) forget poor early choices and avoid premature convergence (c) increase visibility (d) guarantee optimality
- Q39.** Which method keeps a single current candidate?
 (a) GA (b) ACO (c) Simulated Annealing (d) Beam search
- Q40.** In the 6-variable SAT crossover slide, the true h of $P_1 = 010110$ is:
 (a) 3 (b) 4 (c) 5 (d) 6

Answers and explanations

Q	Ans	Explanation
1	b	Implication graph / strongly connected components.
2	c	The phase transition, ≈ 4.26 .
3	b	Any one true literal suffices.

Q	Ans	Explanation
4	b	Ignores h except to remember the best.
5	b	$1/(1 + e^0) = \frac{1}{2}$ at every T .
6	b	$P \rightarrow \frac{1}{2}$ for all moves.
7	a	Only improvements accepted.
8	b	$1/(1 + e^{0.7}) = 0.33$.
9	c	$\Delta E/T \approx 0$.
10	b	From exploration to exploitation.
11	b	$\Delta E_A < \Delta E_B$.
12	b	High T ignores the wiggles and follows the trend.
13	c	<i>Creation: Life and How to Make It</i> .
14	b	Holland (1975); Goldberg popularised them.
15	b	The genotype is the set of genes.
16	b	Fitness-proportional.
17	b	$Nf_i / \sum f = f_i / \bar{f}$.
18	b	$4 \times 576 / 1170 = 1.97$.
19	b	Rare changes restore lost genes.
20	c	The middle bit is 0 in every member.
21	b	Duplicates L, B, J in the slide's example.
22	b	Whole cycles copied from one parent.
23	b	$H \rightarrow A$, $M \rightarrow K \rightarrow D$ in the slide's example.
24	b	Order from the other parent.
25	b	Successor of city i .
26	b	One per direction.
27	b	$A \rightarrow B \rightarrow A$ closes early.
28	a	Every entry stays in its legal range.
29	a	O is 15th, D 4th, G 6th (after removing D), L 10th.
30	b	Resurrection.
31	b	> 3 : overcrowded.
32	b	$\log 3 / \log 2$.
33	b	Dorigo.
34	c	$\eta = 1/d$.
35	b	Only visibility (distance) matters.
36	b	Shorter tours deposit more.
37	b	Evaporate then add deposits.
38	b	Bounds pheromone; forgets.
39	c	One node plus bestNode.
40	c	Clause 1 is satisfied by $e = 1$; the slide's 4 is an error.

Next (per the syllabus): finding optimal paths — Branch & Bound, Dijkstra's algorithm and A*, where the cost already incurred matters as well as the heuristic estimate.